

Distributed Systems, Lecture 01

Foundations and system models

Faculty of Informatics, Juraj Dobrila University of Pula

assoc. prof. Nikola Tankovic, PhD

1 Learning outcomes

After this lecture, you should be able to:

1. Identify the processes, messages, and shared goal in a distributed application.
2. Explain a reason for distribution and the coordination cost it introduces.
3. Distinguish concurrency from parallelism using an execution timeline.
4. Explain partial failure and why a timeout does not establish a remote operation's outcome.
5. State basic network, node, and timing assumptions, and identify a rule the system must always obey.[1]

Prerequisites: basic programming, functions, processes, and an informal understanding of a network. No previous knowledge of distributed algorithms is assumed.

Lecture scope: these foundations support the later lectures on architectures, RPC, clocks, replication, and consensus. We introduce the questions those mechanisms answer; we do not yet implement them. The examples below describe an imagined university project-submission service, not the university's actual infrastructure.

[1] Such a rule is called an **invariant**. For example, retrying the same project submission must never create a second accepted entry. Recall **idempotency** from Web Apps: repeating an operation has the same effect as doing it once. Making submission retries idempotent helps preserve this invariant; the reply to each attempt need not be identical.

2 Example: did my submission succeed?

A student is signed in to a university project-submission portal and already enrolled in the course. For an assignment, they have built a small game in Unity. They package all the project files into a single **ZIP file**: the source code, scenes, and resources such as images and sounds. This **20 MB project archive** is the file they want the teacher to receive. They select it in the portal and press **Submit**. After waiting, the browser displays a timeout. Has the portal accepted the project for the teacher? Should the student submit again?

Uploading bytes and accepting a submission are different steps. Even a progress bar showing 100% does not prove that the portal has validated the rules, saved the complete file, and recorded an accepted submission. In our example, success means that the complete project and its submission record are stored and available for the teacher to retrieve; it does not mean the teacher has opened or graded it.

There are several possible explanations. The complete submission request might never have reached the service; an interrupted upload may leave only temporary bytes. The service might have received it but stopped before storing anything. It might have stored the submission and then lost its reply. Or the request might still be waiting in a queue. The same visible symptom can therefore correspond to different internal states.

This is our central problem for the lecture: **a process must make decisions using incomplete information about other processes**. We will return to the question after building a model of the system. The example develops the uncertain-outcome problem discussed in [3, section 2.1].

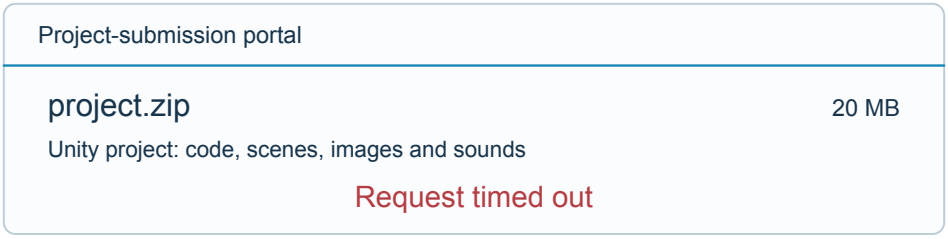


Figure 1: The hypothetical portal reports a timeout without an acceptance receipt.

2.1 A system boundary

The student browser sends the project to a submission service. The **assignment service** supplies assignment details, deadlines, and permitted file types and sizes. The submission service checks those rules and saves the complete project and an accepted-submission record in **submission storage**. We assume the student is already signed in and enrolled, so authentication and enrolment checks are outside this introductory diagram. We include these four cooperating participants in the example's system boundary.

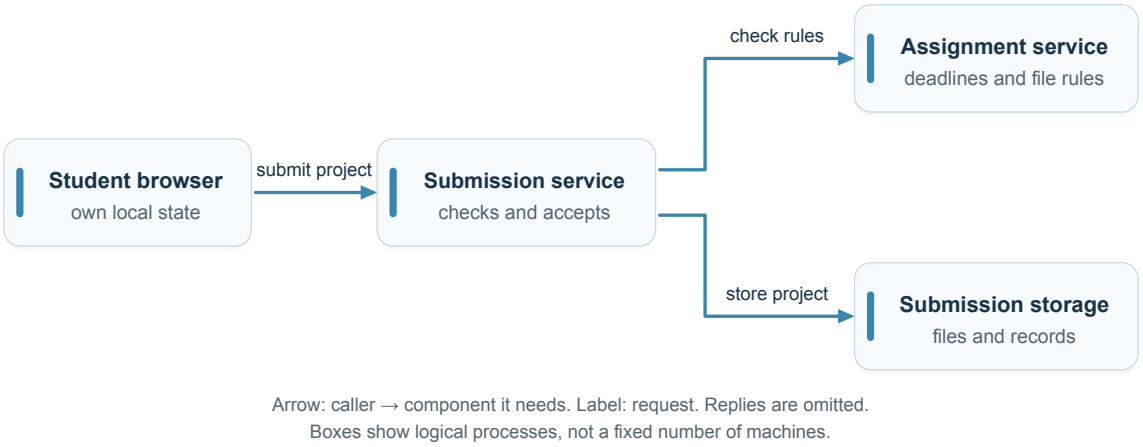


Figure 2: The submission example contains separate processes connected by requests. Each box owns local state. An arrow points from a caller to a component it depends on for this operation; its label names the request. Replies are omitted.

A **process** is a running program. A **node** is a unit that we treat as an independent participant in a particular model: it may be a process or a machine. We must say which meaning we intend. Several processes can share one physical host, so a diagram with four processes does not imply four separate machines or four independent hardware failure domains.

Read each arrow as **"calls and depends on"**: the student browser calls the submission service, and the submission service needs the assignment service to check the rules and submission storage to save the project. The arrow points towards the component being called; it does not mean that information flows only one way. Replies travel back but are omitted from this overview. These are dependencies for the submission operation shown, not a map of every possible dependency in the system.

A submission service cannot simply read a variable in a remote assignment service process as if it were a local variable. In our model, it asks for information by sending a message. Even when an API hides that exchange, information still has to cross a communication boundary.

3 What makes a system distributed?

We will use the following definition throughout this lecture.

Definition 1.1

A **distributed system** consists of independently executing components that communicate and coordinate through message passing to achieve a shared goal.

The application may aim to present a coherent service to its users, even though the work happens in several places. This separates the structure of the system from a useful design goal: hiding unnecessary details of distribution [3, section 1.1].

Three questions help apply the description:

- **Who acts independently?** Identify the participating processes and their local state.
- **How do they communicate?** Identify requests, replies, events, and the boundaries they cross.
- **What are they trying to achieve together?** State a user-visible operation or shared goal.

A folder containing several unrelated programs is not enough to demonstrate a distributed application. Conversely, a single product name or a single web address does not imply that only one process is involved.

3.1 Examples and counterexamples

A browser, API, and database running as separate processes on the same machine coordinate through messages to serve requests. They satisfy our component-based definition, but share a host failure domain. Computers assigned different frames of one film also coordinate toward a shared result and satisfy our definition. Three unrelated laptops on the same Wi-Fi do not: network access alone supplies neither coordination nor a shared task. Threads communicating only through shared memory illustrate concurrency, but do not satisfy this definition's message-passing criterion.

An API and worker in separate containers on one laptop satisfy our component-based definition when they coordinate through messages. They still share a host failure domain; the deployment does not tolerate loss of that machine. Some definitions reserve "distributed system" for multiple computers, so state the definition being used.

For a concrete message-passing example, **three MPI processes calculate the average temperature in a large dataset**. Process 0 divides the measurements into three chunks, keeps one, and sends the others to processes 1 and 2. Each process computes a local sum and count. The workers send their results to process 0, which calculates total sum divided by total count. The processes act independently and coordinate through messages toward a shared goal. They may run on one computer or several; that changes the failure domains, not the message-passing structure of this example.

3.2 Coherence and transparency

The submission system should offer a meaningful service: students select an assignment and upload a project and obtain an understandable result. They should not need to choose a database machine or understand the assignment service's deployment. **Distribution transparency** means hiding selected implementation details from the user or caller.

Transparency is not a promise that communication has no cost or that failures can always be hidden. For example, replacing an uncertain result with a confident "Submission failed" is misleading if the submission might already exist. A useful interface may instead show "Outcome unknown; checking status" and offer a way to retrieve the result. The policy must match what the system actually knows [2].

We will not require complete transparency for every application. An offline indicator or an explicit synchronization status can be helpful information rather than a design defect.

4 Why distribute the work?

Three reasons account for most distributed systems. First, **the components are physically apart**: users, devices and data sources are where they are, and the system must reach them. Second, **one machine does not have enough capacity** for the work. Third, **the system must keep working when a part fails**, which requires redundant copies of services and data. Because nearly every information system faces at least one of these needs, nearly every information system is distributed. Even a simple web application runs a browser, a server and a database as separate processes, often on separate machines.

Reason	**Submission-service example**	**New question to answer**
Components are physically apart	Students submit from home, campus or abroad; sensors and phones in other systems send data from where they are.	How long do messages take, and what happens when a connection drops?
More capacity is needed	Before a deadline, several workers process different projects.	How do they avoid accepting the same submission twice?
The system must survive failures	Copies of the service and of stored projects (redundancy) let work continue when one stops.	Which state survives, and how does a client find the replacement?

Distributed-system design also pursues further goals; the textbook in [1] discusses them in more depth. **Resource sharing** lets multiple courses reuse storage or assignment rules. **Transparency** gives students one coherent portal without requiring them to choose a worker. **Openness** means components follow agreed interfaces so, for example, a faculty assignment service run by another team and the submission portal can cooperate; it does not necessarily mean open-source software. These are design goals to pursue, not automatic consequences of adding machines, and none removes the need to reason about coordination [1].

Scaling up means increasing the capacity of an existing component. **Scaling out** means adding components. Scaling out can increase capacity when work can be divided effectively,

but a shared bottleneck or additional coordination can limit the benefit. Adding servers does not automatically make an operation correct or faster.

For a small application, one well-understood process and submission storage may be adequate. Distribution becomes justified by a concrete requirement, not by the number of boxes in an architecture diagram. The cost includes communication, deployment, recovery, and reasoning about states that different components can observe at different times.

4.1 Facebook: scale as a motivation

Facebook reported **1 billion monthly active users in October 2012**, **2.50 billion in December 2019**, and **3.07 billion in December 2023**. These are monthly activity counts, not simultaneous users or requests per second. Facebook stopped reporting standalone daily and monthly activity metrics from Q1 2024; newer Meta Family figures combine products and are not a continuation of this series. Sources: 2012 milestone, 2019 results, 2023 results, and reporting change.

Growth in users and their activity motivates distributing computation and data when one machine's resources no longer suffice. Facebook's engineering account of scaling discusses partitioning and caching among its techniques. Those mechanisms introduce coordination and recovery work. This lecture introduces scaling up and out; Lecture 11 develops partitioning, load distribution, caching and bottlenecks. Asynchronous communication belongs to Lectures 03 and 10, and replication to Lecture 06.

5 Must everyone wait for one submission?

Imagine a large university where many students submit their projects just before the deadline. Receiving the ZIP file is only the first step. The portal checks assignment rules, unpacks and scans the files, saves the project, and records its acceptance. Suppose these steps take **20 minutes for one submission**.

Does this mean nobody else can submit during those 20 minutes? Not necessarily. We need to know how the work is organized:

- With **one worker processing one complete job at a time**, later jobs wait in a queue. With a fixed 20-minute processing time and continuous work, that worker completes three jobs per hour. Other students may still upload their files if receiving uploads is handled separately.
- If much of those 20 minutes is spent **waiting for storage or another service**, a worker may process parts of other submissions during those waits. Several jobs can be in progress without executing instructions simultaneously.
- With **several workers and enough resources**, different projects may be processed at the same time. Two independent workers could ideally complete six such jobs per hour, but only if shared storage, the network, and other checks do not become bottlenecks.

So the time for one submission does not determine how many submissions the whole system can complete per hour. **How can we complete more work in the same amount of time while still accepting each project correctly?** The rate of completed work is called **throughput**. Reducing one student's waiting time is another goal. The next section names these different ways of organizing work.

6 Concurrency and parallelism

Let A and B be different students' submissions. A worker can progress B while A waits for storage, or two workers can process A and B simultaneously. These alternatives answer the workload question and motivate the following terms. Their instructions need not execute at the same instant: one worker may switch between them. With two processor cores, both tasks may actually execute at the same time [2].

Definition 1.2

Concurrency means that multiple tasks are in progress during overlapping periods.

Definition 1.3

Parallelism means that multiple tasks execute at the same time.

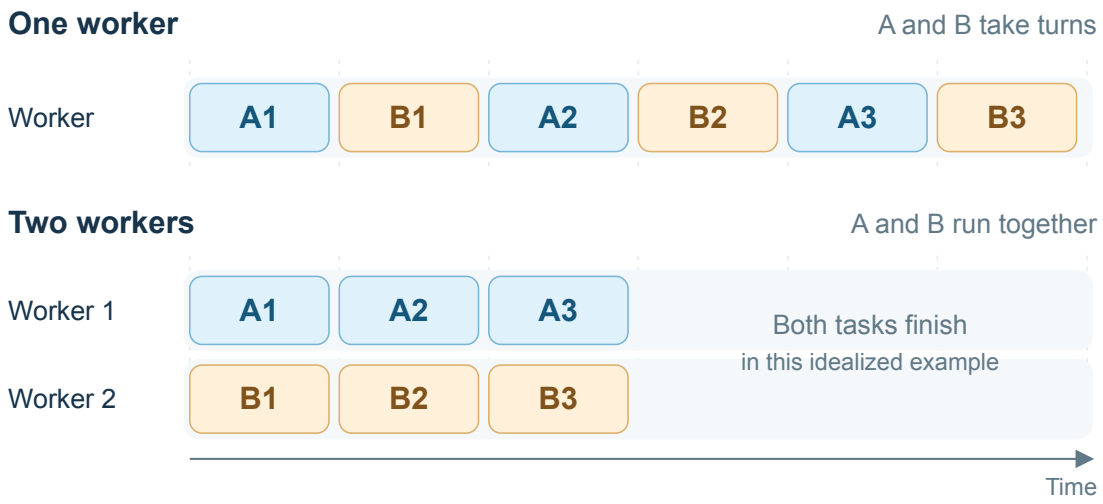


Figure 3: The upper timeline interleaves tasks A and B on one worker. The lower timelines execute A and B on separate workers at the same time. Each labelled block is a piece of a task, not necessarily one instruction.

Recall the 20-minute submission checks: that duration forced three jobs per hour only under the assumption of one worker processing jobs strictly one at a time.

Overlapping work can keep resources busy during waits, and parallel execution may increase completed jobs per hour when shared resources keep up. We will later reuse this execution pattern to reason about two attempts of the same submission.

Concurrency can occur inside a single machine; it does not require a distributed system. Distribution adds communication between independently executing participants. Parallelism

can also occur inside a single machine, so multiple cores are not by themselves evidence of a distributed application.

Under our definition, parallel tasks also overlap, so parallel execution is concurrent; concurrent tasks do not have to run in parallel. A single worker may switch on a wait, a yield or scheduler preemption. Dependencies still constrain the order of steps. Concurrency does not mean arbitrary reordering preserves the result: unsynchronized shared state can introduce races.

6.1 When does a worker switch tasks?

A task can switch in three ways (Figure 4). First, the task **waits for I/O**, such as storage, and the worker meanwhile continues another task. Second, **the operating system preempts the thread** when its time slice expires; the thread does not choose this and does not know where it will stop. Third, a task can **yield voluntarily** without any I/O.

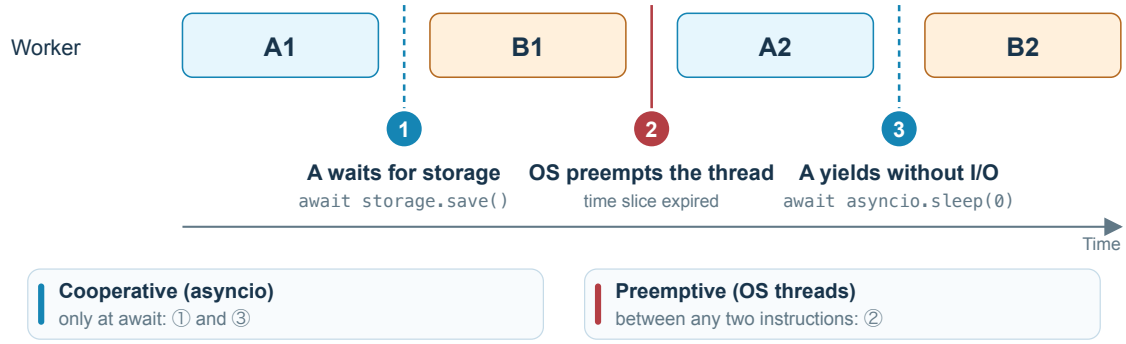


Figure 4: Three task-switch points on one worker. Cooperative scheduling (asyncio) switches only at `await`; preemptive scheduling of operating-system threads can switch between any two instructions.

In `asyncio`, one thread runs several coroutines, and a coroutine switches only at `await`. The operating system can still pause the whole process, but then no other coroutine of the same event loop runs. A long CPU-only loop therefore holds the event loop; `await asyncio.sleep(0)` waits for nothing and only yields to other tasks:

```
async def check(submission):
    for file in submission.files:
        scan(file)                  # CPU only: the loop cannot switch tasks
        await asyncio.sleep(0)      # voluntary switch point, no I/O
    await storage.save(submission) # waiting for I/O: switch point
```

With threads, the operating system preempts execution between any two machine instructions, and CPython switches threads between bytecode instructions (every 5 ms by default; see `sys.getswitchinterval()`). So even `counter += 1` is not indivisible: read, add and

write are separate steps. Either way, **someone else can run between the check and the write.** We return to this with retries.

The same logic applies on two levels (Figure 5). The operating system schedules threads on a core preemptively. Inside one thread, the event loop schedules coroutines cooperatively: it is an ordinary loop in user space, not a separate process. A task (`Task`) corresponds to a thread, the loop's ready queue to the ready queue of threads, and `await` on I/O to a blocking call: the socket is registered, and the coroutine resumes when `epoll` reports it ready. There is no preemption at the loop level, so a coroutine that never reaches `await` stops all others. When the operating system pauses the thread running the loop, neither the loop nor any coroutine runs, so the coroutine does not notice the pause. Code between two `await`s is therefore indivisible for other coroutines of the same loop, but not for other threads or processes.

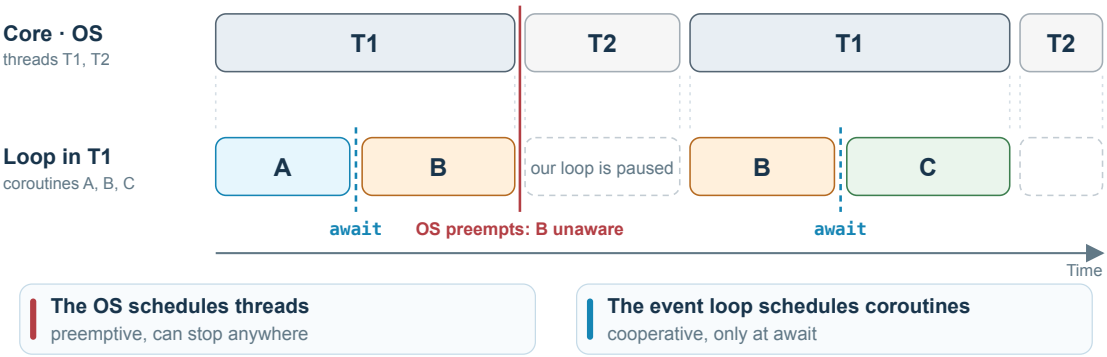


Figure 5: Two schedulers. Above, the operating system alternates threads T1 and T2 preemptively; below, the event loop in thread T1 switches coroutines only at `await`. While T2 runs, the loop is paused.

Rob Pike expresses the distinction as: "Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once." See *Concurrency is not Parallelism*. A cook managing two dishes illustrates switching attention; two cooks can perform actions simultaneously. This analogy concerns active attention, not every physical process in the kitchen.

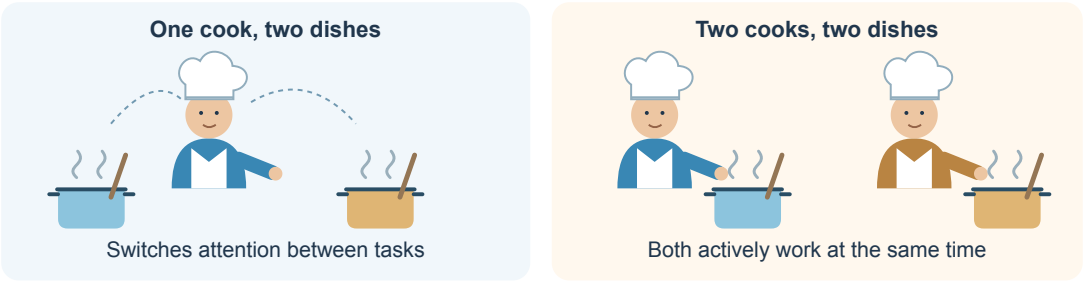


Figure 6: One cook switches attention between two dishes; two cooks can actively work simultaneously.

Overlapping useful work with a remote wait hides some waiting from the caller; it does not remove network latency or guarantee higher throughput.

Pause and predict: if job A is waiting for storage, can the worker progress job B? Does this require two processor cores?

7 A remote request has more than a return value

A function-call interface can hide a remote exchange. Underneath it, a caller prepares a request, the request travels to another process, that process performs work, and a reply travels back. For a worked introduction to this boundary and the conversion between local values and messages, see [3, section 1.3].

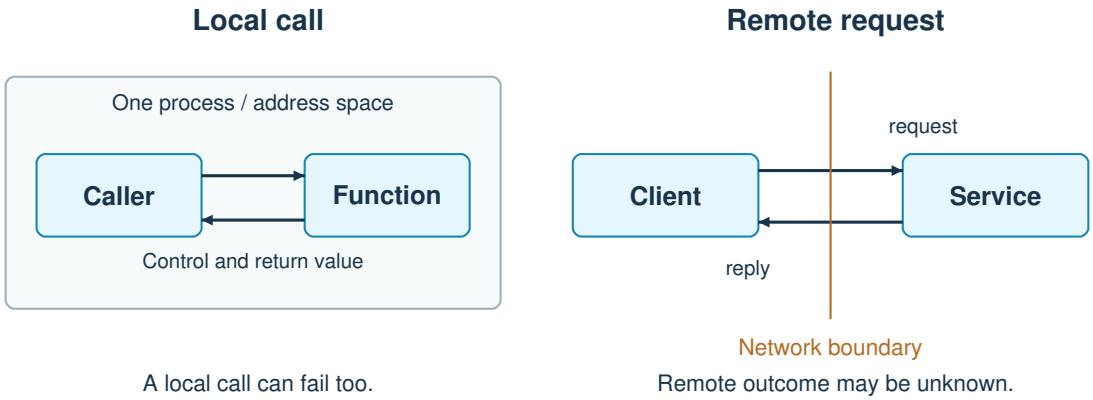


Figure 7: A local call transfers control within one process. A remote request crosses a network boundary in both directions, introducing uncertainty about whether work happened when no reply arrives. Local calls can also fail; the distinction is not that they are infallible.

A remote call has several possible costs: request preparation, transmission, queueing, processing, and reply handling. These costs are not necessarily constant. A service that normally responds quickly may pause, become overloaded, or wait for another dependency.

The difference is clear in code. A local deadline check has two outcomes, a value or an exception, and it ran exactly once:

```
def check_deadline(task, now):
    return now <= DEADLINES[task]        # KeyError if the task does not exist
```

The same check as a remote request must distinguish more cases, and one of them has no answer:

```python
async def check_deadline(client, task, now):
    for attempt in range(3):
        t0 = time.perf_counter()
        try:
            r = await client.get(f"/tasks/{task}/deadline", timeout=2.0)
            r.raise_for_status()
            return now <= datetime.fromisoformat(r.json()["deadline"])
        except httpx.ConnectError:
            pass   # request not sent: safe to retry
        except httpx.TimeoutException:
            pass   # slow or crashed? a read is safe to retry
        except httpx.HTTPStatusError as e:
            if e.response.status_code < 500:
                raise  # 404: retrying does not help
        except (KeyError, ValueError):
            raise      # different reply: another service version
        finally:
            ms = (time.perf_counter() - t0) * 1000
            log.info("deadline %s, attempt %d: %.0f ms", task, attempt + 1, ms)
        await asyncio.sleep(0.2 * 2**attempt + random.uniform(0, 0.1))
    raise RuntimeError("task service not responding")
```

We must also decide how often to retry and how long to wait between attempts: exponentially growing waits with random jitter keep many clients from retrying at the same moment. Logging the duration of every attempt (`finally`) shows whether the service is slow or unavailable. A local call needs none of this. The deadline check only reads, so a retry changes nothing. A write is different: if waiting expires after `POST /submissions`, we do not know whether the submission was stored. We return to this in the section on three histories behind one timeout.

A chain of calls: reliability multiplies. A submission rarely calls just one service. The deadline check, user data, storage, and notification are already four remote calls, and the submission succeeds only if all of them do. If each of N sequential steps succeeds independently with probability p and there are no retries, the whole workflow succeeds with probability

$$P_{\text{success}} = p^N, \qquad P_{\text{at least one failure}} = 1 - p^N.$$

Table 1: Probability that a workflow of N sequential steps succeeds, with independent failures and no retries.

Number of steps N	$p =$ **99.99%**	$p =$ **99.9%**	$p =$ **99%**	$p =$ **98%**
1	99.99%	99.9%	99%	98%
4	99.96%	99.60%	96.06%	92.24%
10	99.90%	99.00%	90.44%	81.71%
100	99.005%	90.48%	36.60%	13.26%

With only four calls at 99%, one submission in twenty-five gets stuck somewhere. Over a hundred steps, the difference between 99.99% and 98% per step becomes the difference between 99% and 13% for the whole workflow. Conversely, a target for the whole workflow sets a requirement for each step: $p_{\min} = P_{\text{target}}^{1/N}$. For 99.9% over 100 steps, each step must succeed at least 99.999% of the time, that is, fail only about ten times per million calls. Latency behaves the same way: if 1% of one service's replies are slow and a request waits for replies from a hundred services, about 63% of such requests are slow [4]. The assumptions are simplified. Real failures are often correlated (a shared network, a shared database), and a retry raises a step's p but adds load. We return to this with retries.

Latency describes a delay, such as the time from issuing a request to receiving a response. **Bandwidth** describes a communication link's data-transfer capacity. **Throughput** is the rate of completed work, such as accepted projects per second. A link with high bandwidth can still have high latency; a service with high throughput can still have slow individual requests. State the measurement and its units rather than treating these terms as interchangeable [3, section 1.2]. A 20 MB archive makes transfer time noticeable, but validation, queueing, and storage can also delay confirmation. The file size is illustrative, not a measured performance claim.

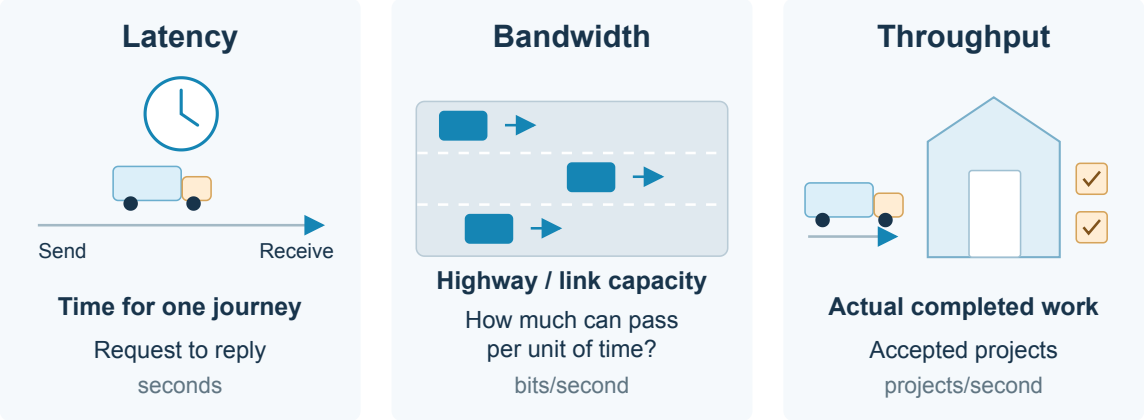


Figure 8: A delivery analogy separates the time for one delivery, the route's cargo capacity, and the rate of completed deliveries. These measure different things: a large cargo capacity does not by itself guarantee a short trip or fast acceptance at the destination.

Imagine sending the project as a parcel. **Latency** asks how long this delivery takes; **bandwidth** asks how much cargo the route can carry per second; **throughput** asks how many deliveries are actually completed per second. Cargo volume and completed deliveries are different units. In the portal, we measure latency until the result reaches the student, including the reply. We count throughput here as accepted projects per second. A fast upload can still be followed by a wait for validation or storage: the file arriving is not the same as the project being accepted.

Thinking question: how would you transport **100 TB of data between two datacenters as fast as possible**? What information would you need before choosing a method?

Suggested reasoning: compare sending the data over the network with copying it to drives and transporting them. Define completion as all data copied and verified at the destination. Ask about distance, sustained network transfer rate, available drive capacity, copying speed, and transport time. Using decimal units, 100 TB contains 800 trillion bits; at a sustained 10 Gbit/s, transfer alone takes about **22.2 hours**, before additional overhead. For physical transport, include writing the drives, travel, reading them, and verification. Neither method is always fastest. Moving drives can offer high effective throughput for a large batch while having long delivery latency; it would be a poor way to obtain a quick reply to a small request. These numbers are illustrative, not measured datacenter performance. Returning to the portal: moving lots of data efficiently does not guarantee a quick response to one student.

7.1 Recall Web Apps: time to first byte

Time to first byte (TTFB) measures waiting until the first response byte, rather than completion of the response download. Here we measure from navigation start, including DNS and connection setup when needed. A measurement starting at the HTTP request excludes that earlier work. Always state the starting point. The wait includes network travel, server queueing and processing. See web.dev, Time to First Byte.

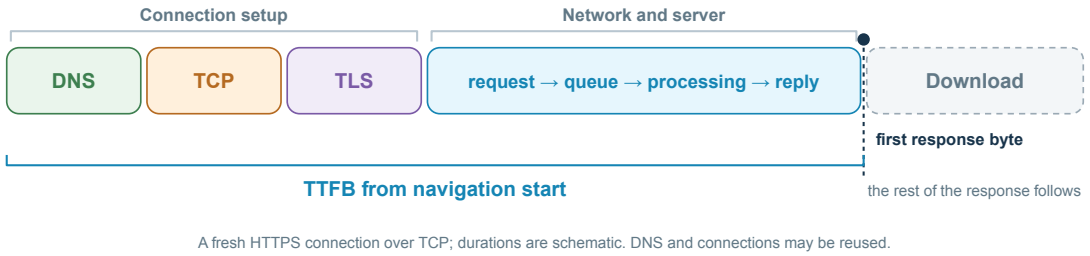


Figure 9: A simplified fresh HTTPS connection over TCP. TTFB ends at the first response byte; downloading the remaining response follows. DNS and connections may be reused. Segment widths are schematic.

High bandwidth does not remove waiting for server work or network travel. Download time depends on response size and achieved transfer rate. In our portal, an early response byte does not prove durable acceptance: the response might only acknowledge pending work. This sketch omits redirects, cache and service-worker handling, and informational responses; HTTP/3 uses QUIC rather than the illustrated TCP setup.

8 Partial failure

The portal still opens, but the assignment service has stopped. Can a student retrieve an old receipt? Can a new submission be accepted without checking the deadline? An assignment service can stop responding while the submission process, submission storage, and student browser remain alive. This concrete contrast motivates the following term.

Definition 1.4

A **partial failure** occurs when some components fail while others continue running.

Components need not fail together, and a live process can still be unable to complete a particular operation [2; 3, section 2.3].

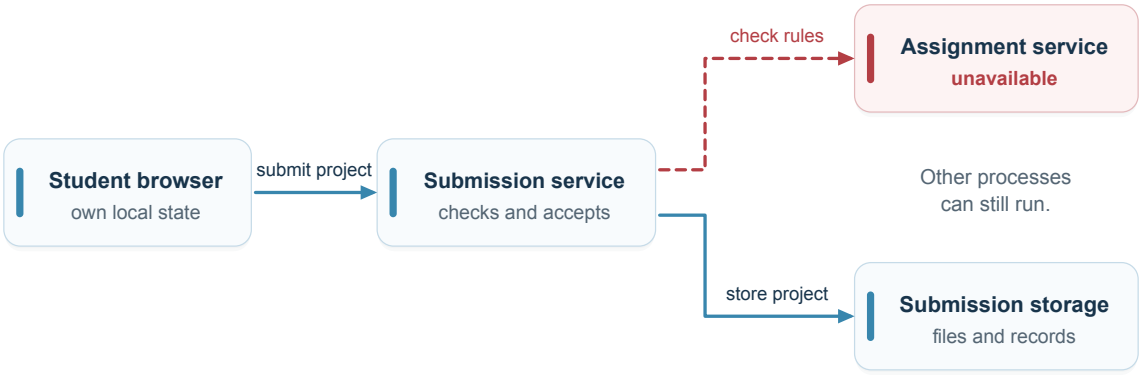


Figure 10: The assignment service is unavailable, while the other processes continue running. Whether submission can complete depends on the information and guarantees the operation requires.

"Server is running" and "service operation is available" are different statements. If every submission must check the current deadline and file rules, losing the assignment service can prevent new acceptances even while uploads or receipt retrieval still work. Another operation, such as reading a previously stored receipt, might still work. Availability must be discussed for a particular operation and its required guarantees.

A network interruption can also prevent two live processes from communicating. An observer that receives no reply cannot always tell whether a process stopped, the route failed, or the reply is merely delayed. Failure detection is therefore connected to the assumptions we make about time.

9 Three histories behind one timeout

9.1 Who is the client, and why use a timeout?

A **client** is the caller in an interaction; it need not be a browser. The submission service acts as a server when it receives the browser's request, and as a client when it asks the assignment service to check rules. Each caller may configure its own waiting deadline through application or library settings, so a call does not leave it waiting indefinitely. These deadlines are not automatically coordinated across tiers.

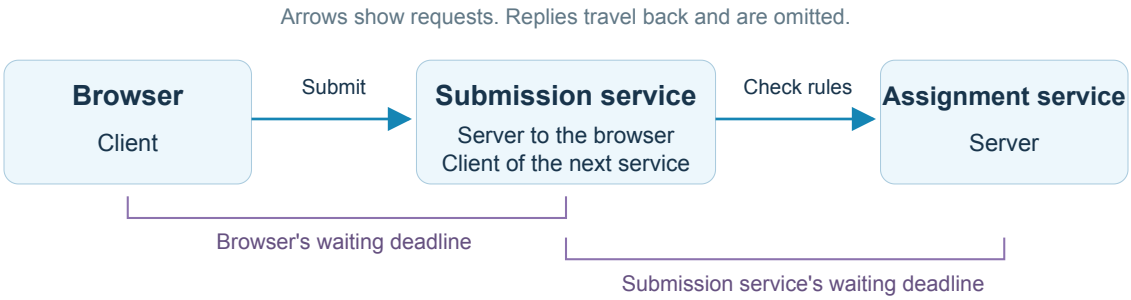


Figure 11: Client and server are roles in each call. The middle tier is a server to the browser and a client of the assignment service. Each caller can set its own waiting deadline.

A timeout means the caller gave up waiting. It does not destroy the request: that request may still be travelling, queued, or running. Nor does a timeout automatically cancel remote processing, undo a stored effect, or establish the outcome. Explicit cancellation needs a separate mechanism and may race with completion. Here a request is an application-level operation that may span many network packets. The following histories use a browser as the client, but the same uncertainty occurs between services.

9.2 Three possible histories

Return to the student's request. We compare three histories with the same client-visible result. The arrows summarize a whole submission operation, not one network packet. History A includes an upload that never completes; temporary fragments are not accepted submissions. History B stores the complete file and accepted record before its reply is lost. History C stores them durably before the server process crashes without sending a reply.



Figure 12: History A: the complete submission request never reaches the service. No submission is accepted, and the client eventually reports a timeout.



Figure 13: History B: the service receives the request and stores the submission, but the reply is lost. The client reports the same timeout as in history A.

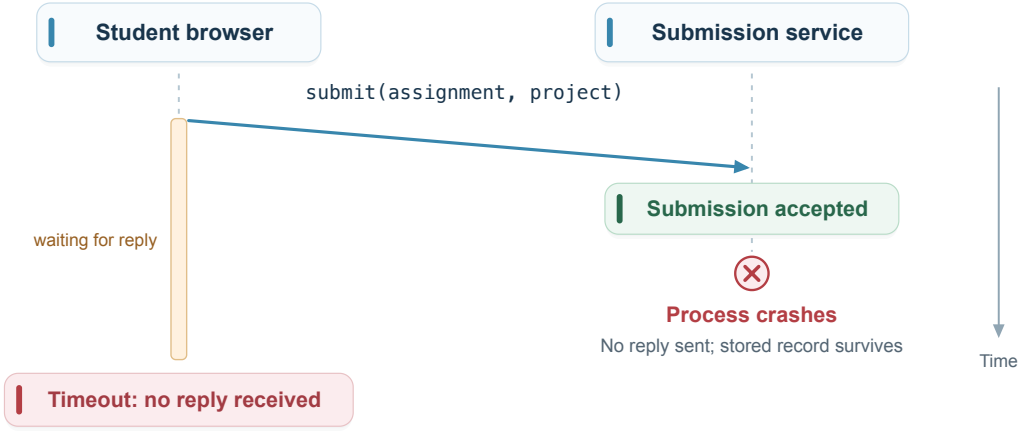


Figure 14: History C: durable acceptance precedes a server process crash. No reply is sent and the browser times out; the accepted record survives.

History C assumes the process failure does not destroy the durable submission record. A crash before acceptance could instead leave no accepted submission. We depict a client receiving no conclusive response before its deadline; a transport may sometimes report a connection error sooner.

In all three histories, the client sent a request and did not receive a reply before its deadline. The server-side effect differs. A **timeout is an observation about waiting**, not a definitive statement that a remote effect did or did not happen.

These are not the only histories. A request might still be in flight, queued, or partly processed when the timeout fires. The examples show why the observation alone cannot distinguish success from failure. They apply the uncertain-outcome concept in [3, section 2.1] to project submission.

9.3 Discussion: should the student retry?

A retry could help in history A. In histories B and C, an unprotected retry might repeat an effect, such as creating a second submission entry or notifying the teacher twice. History C requires recovery before the service can handle the retry. Whether duplication happens depends on the operation and implementation; a timeout does not itself imply duplication.

A better design asks how a client can refer to **the same intended operation** across attempts. For example, a request identifier can be associated with a recorded result. A repeated request can then be recognized. However, generating an identifier alone is insufficient: the receiver must use it consistently, and crashes must not leave the effect recorded without the information needed to recognize it.

We will develop retry semantics, idempotency, and durable duplicate detection in later lectures. For now, retain the design question: **how can the application resolve an uncertain outcome without accidentally repeating an effect?**

9.4 Reusing concurrency to reason about retries

Not everything is atomic in distributed systems. In this example, checking storage and creating the accepted record are separate steps. Between them the service may wait for I/O, yield to another request, or be paused by the operating system. Even a short gap can allow another attempt to act on the same earlier state. Atomicity would make the relevant operation indivisible with respect to competing operations; it must be provided by a suitable mechanism, not assumed. The same check-then-write race can occur on one machine.

Consider another possible history: the original request is slow or queued rather than lost. The client sends attempt A, its waiting deadline expires, and it sends retry B. The server may now be working on both attempts for the same intended submission. A tempting implementation is "if no accepted entry exists, create one". What happens if both attempts perform the check before either creates an entry? The following interleaving exposes the problem.

A timeout does not cancel work already running at the server. Recall the earlier concurrency diagram, but now let A and B be attempts of the same submission rather than different students' work. Suppose these two attempts represent the same intended project submission. A checks for an existing accepted submission and finds none. Before A records its result, B also checks and finds none. If both then create a new accepted-submission record, the teacher sees two entries for the same intended submission. A retry can overlap the original attempt even after the browser has timed out.

This history does not require simultaneous execution. An interleaving on one worker can expose the same logical problem when an operation spans several steps. We therefore reason about permitted interleavings and shared state, rather than assuming that a fast processor or a single service instance removes the problem.

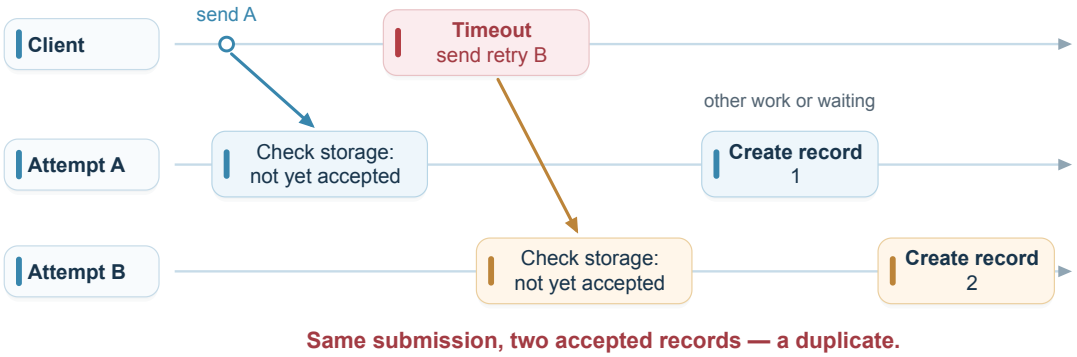


Figure 15: The client times out and retries while attempt A continues. Both attempts check stored submissions before either records acceptance, then create two accepted records for one intended submission.

10 Invariants and progress

The teacher sees two accepted entries after a retry. What rule should have prevented this? We need a condition that holds across every permitted interleaving, including two overlapping attempts.

Definition 1.5

An **invariant** is a rule that must hold in every permitted state of the system.

For the project example, a useful invariant is "retries of the same intended submission create at most one accepted entry." We choose this rule for the example. A deliberate revised project can be a new version under an explicit resubmission policy; it is different from retrying the same operation after a timeout.

Suppose we fix duplication by rejecting every submission. We now avoid duplicate entries, but students can never complete their work. A useful portal needs both protection from bad outcomes and a promise of progress under suitable conditions.

Definition 1.6

A **safety property** requires that a specified bad event or state never occurs.

For example, the service must never accept duplicate entries for the same intended submission. Maintaining the one-entry invariant supports this safety property.

Definition 1.7

A **liveness property** requires that specified progress eventually occurs under stated assumptions.

For example, a valid submission eventually gets accepted, provided the necessary services recover and remain available, communication eventually succeeds, pending work is retained or retried, and queued work is eventually processed. Validity follows the applicable assignment rules, including the deadline policy. This is a progress guarantee, not a promise of a particular response time. The exact property and assumptions must always accompany a correctness claim.

A service can avoid duplicates by refusing every submission, but that is not a useful complete solution. It can respond quickly by creating an entry for every request, but retries may then produce duplicates. We need to state both what must not go wrong and the conditions under which useful work must complete.



Figure 16: Two distinct failures: duplicate acceptance and no eventual acceptance despite the progress assumptions holding.

Before choosing a protocol, write down the invariant, the allowed failures, and the evidence available to each participant. Those three items make an architecture discussion much more precise than a list of technologies.

10.1 Safety and liveness in different systems

System	Example safety claim	Example liveness claim
Payments	Retries never charge twice for the same intended payment.	A valid, authorised payment eventually completes.
Seat booking	Two customers never hold confirmed bookings for the same seat and event.	Every request eventually receives confirmation or rejection.
Job queue	A job is never marked complete before it succeeds.	Every valid pending job is eventually processed.

System	Example safety claim	Example liveness claim
Distributed lock	At most one process executes the protected critical section at a time.	Every correct process that keeps requesting the lock eventually acquires it.

These are claims we might require, not properties automatically supplied by the system name. Progress requires appropriate service recovery, successful communication, preserved or retried work, and fair scheduling. Jobs must terminate or reach a defined failure outcome. The lock claim is starvation freedom and additionally requires fair granting, finite critical sections and safe recovery from failed holders. Booking rejection satisfies the stated response claim; it does not promise everyone a seat. Queue safety here does not assert exactly-once execution. As before, eventual progress does not provide a numerical response-time bound.

11 State the system model

Can we promise every valid submission will eventually be accepted if the power stays off, the disk fails with no recoverable copy remaining, or a cut cable leaves the service permanently disconnected? These examples challenge liveness: a stopped system may avoid duplicate acceptance but cannot make progress. Temporary failures may be tolerated when power and connectivity recover and work survives or is retried. A disk failure need not lose work if a replica, backup or client copy allows recovery; the example assumes no such recovery or resubmission remains possible. Our claims depend on assumptions about power, storage, communication and recovery. The next question is: **under what failures and delays can the system provide its claimed properties?**

A **system model** makes assumptions about participating nodes, communication, and timing explicit. It is an abstraction used to reason about behaviour; it is not a drawing of every implementation detail [3, section 2.3].

For example, we may model the portal as allowing delayed or lost messages, a submission service that can crash and restart, and no guaranteed maximum reply time. An algorithm makes **correctness claims**, and each claim must hold in every execution allowed by its stated model assumptions. One example claim is: **retries never create two accepted entries for the same submission, even after a service restart**. This is a property to justify from the algorithm and its assumptions; the model alone does not provide it.

A progress claim, such as “a valid submission is eventually accepted”, needs further assumptions: services recover and remain available, communication eventually succeeds, pending work survives or is retried, and processing eventually runs. Allowing messages to be lost forever would prevent that progress guarantee.

11.1 Network assumptions

We make infrastructure assumptions explicit through network behaviour, node behaviour and timing. Start with the cut cable: does communication eventually recover, or remain broken? Retrying is useful only if some later attempt can reach the receiver. This is the network part of the model; node failures and timing bounds follow below.

What if every retry is lost? Retrying a missing request is useful only if some attempt can eventually get through. We must state that condition rather than infer it from the act of retrying.

Assumption	Meaning for the example
Messages may be lost, duplicated, or reordered	Handle missing replies, repeated requests, and different arrival orders.
Persistent retries eventually get through between correct endpoints	Eventual delivery becomes possible, but no delivery deadline follows.
A partition blocks communication permanently	Work requiring that communication cannot be guaranteed to finish.

A **fair-loss** link permits loss but guarantees eventual delivery under persistent retransmission between correct endpoints. A **reliable** link provides eventual delivery without duplication or fabrication; it need not preserve message order. An **arbitrary** link may also allow an adversary to modify or fabricate messages. These link models describe permitted behaviour, not a delay bound [3, section 2.3]. We do not develop their implementations in this lecture.

A network partition prevents communication between some participants even while they remain alive. It may be temporary or permanent. Application retries and network duplication are distinct reasons a receiver can encounter repeated requests. Both matter when protecting the one-entry rule.

For our worked example, requests and replies can be delayed or lost. The example does not contain an active attacker. Those are choices for the model, not universal facts about every deployment. If we later assume that retransmission eventually succeeds, we must say so; a permanent partition would contradict that assumption.

A delivery guarantee and a timing guarantee are different. “Eventually delivered” does not imply “delivered within two seconds.” We should not infer a deadline from a statement that only promises eventual progress.

11.2 Node assumptions

The service stops after accepting the project. What happens when it returns? First ask which state survives. An accepted record in durable storage and a pending task held only in memory have different recovery behaviour. Then distinguish restarting, never returning, and departing from the algorithm altogether.

Model	What may happen	Consequence for reasoning
Crash-stop	A participant stops permanently within the model; no recovery is assumed.	The algorithm must cope without relying on its return.
Crash-recovery	A node may restart after a crash.	Specify which state is durable and which in-memory state is lost.
Byzantine	A faulty node may deviate arbitrarily from the protocol.	Surviving crashes alone is not sufficient protection.

For example, a worker dies during a computation and surviving workers must finish without it. Crash-stop excludes further steps by that participant in the execution being analysed; it does not assert that its physical machine can never reboot. If the same participant restarts and rejoins, use crash-recovery and specify which state survives. Finishing despite a failed worker is a requirement to design for, not an automatic guarantee of the model.

These categories describe allowed behaviour, not how quickly a failure is detected. In particular, a crash-stop assumption does not automatically give other nodes immediate knowledge of the crash. We do not analyse Byzantine agreement in this introductory lecture.

In the portal, **crash-stop** could mean the submission process stops and never runs again during the execution being analysed. Under **crash-recovery**, it restarts: assume accepted records survive in durable storage, but its in-memory queue is lost. A **Byzantine** service could tell the student “accepted” and the teacher “not accepted”. Conflicting replies are one example of arbitrary deviation, which need not be malicious.

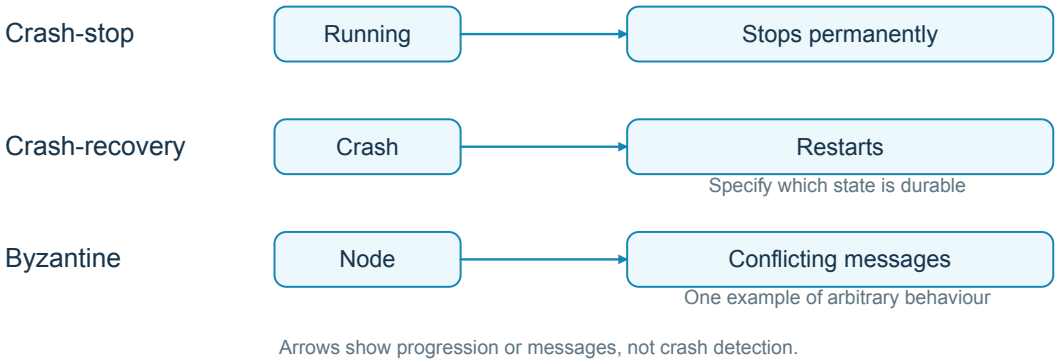


Figure 17: Examples of permitted behaviour under three node-failure models. Arrows do not imply that another node detects a crash.

11.3 Why the timing assumptions matter

The assignment service usually replies quickly. Today it is silent. How long should the portal wait? Should it retry, keep waiting, or tell the student that the outcome is unknown? Here "we" means the portal's system architects and developers: the people who choose and implement the waiting and retry policy. The running service follows that policy. Choosing a timeout does not create a response-time guarantee. The timing assumptions determine what evidence a missing reply provides [3, section 2.3].

Without a guaranteed response-time bound, silence cannot distinguish a slow service from a crashed one. A timeout can still trigger a recovery action, but the original request may remain active. A retry must therefore preserve the no-duplicate invariant even when both attempts overlap. This connects the model directly to **safety**.

Insight: timeout and retry policies can overload the service they are meant to help. If many clients use too-short timeouts and retry aggressively, their original requests may still be running when new attempts arrive. More load causes slower replies, which trigger more retries: a **retry storm**. Legitimate distributed clients can thereby cause a self-inflicted denial of service with DDoS-like effects, without a malicious attacker. Bound the retry count or budget and use backoff with jitter (randomized delays) to reduce amplification and synchronized bursts. These measures control load; they do not replace safe handling of duplicate requests. See AWS Builders' Library, *Timeouts, retries, and backoff with jitter*.

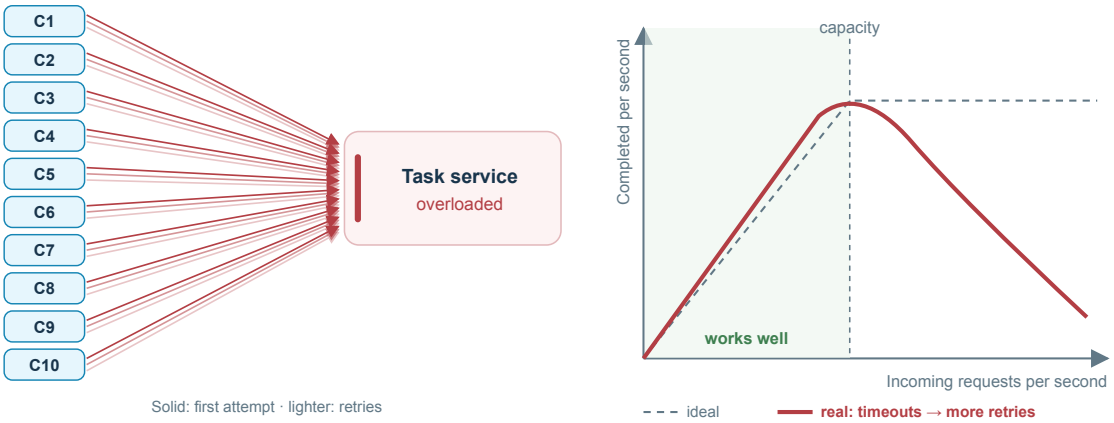


Figure 18: A retry storm. Left: ten clients, each with a first attempt and two retries, send requests to the same overloaded service. Right: a schematic curve: up to capacity, completed requests grow; above capacity they fall because timeouts trigger more retries.

There is also a **liveness** consequence. Waiting forever avoids an overlapping retry but can leave valid work unfinished. Eventual acceptance needs explicit conditions: the necessary services recover and remain available, communication eventually succeeds, pending work is retained or retried, and processing eventually runs. These conditions support a progress argument; they do not supply a fixed response deadline.

If a model guarantees that every live service replies within a known end-to-end bound, with reliable communication and the other assumptions holding, exceeding that bound rules out the healthy execution. A network-delay bound alone is insufficient: processing and the return path also take time. When messages may be lost or the network may fail independently, silence cannot uniquely identify a service crash.

In an eventually synchronous model, timeout-based suspicions can be wrong before stabilization. Once the bounds hold, a timeout longer than the relevant response bound can avoid false suspicions caused by delay, subject to the remaining assumptions. Participants do not know when stabilization happens, and the timing assumption does not itself guarantee that an arbitrary algorithm makes progress. We must still explain how the algorithm uses it.

Thus, a claim such as “the system will eventually accept the submission” must state the assumptions under which it holds. The following models make the timing part of those assumptions explicit.

11.4 Timing assumptions

In a **synchronous model**, suitable known bounds constrain communication and processing delays. In an **asynchronous model**, the algorithm cannot rely on such timing bounds. A common **partially synchronous** model allows an initial period without usable timing

bounds, followed by a period in which bounds hold; the algorithm may not know when that stable period begins.

Why use partial synchrony? Internet congestion, routing changes and pauses in process execution motivate a model that allows unpredictable delays [3, section 2.3]. The figure uses the **eventual-synchrony** formulation: after an unknown finite stabilization time, the bounds hold for the remainder of the execution. This is an assumption for reasoning, not an Internet guarantee or a claim that 99% of requests are fast. Real deployments can repeatedly become unstable; a sufficiently long stable interval may let an operation finish, depending on the algorithm and its other assumptions. A useful design goal is **safety despite unpredictable timing, and progress once conditions stabilize**. The algorithm must establish both claims; selecting the model does not prove them.

The vocabulary here describes guarantees. It does not mean the same thing as a blocking versus non-blocking API. A program using `async` and `await` is not thereby proving that its distributed algorithm uses an asynchronous system model.

11.5 Examples with illustrative numbers

The following numerical examples illustrate the timing assumptions independently of the deployment:

- **Synchronous:** assume every message between healthy nodes arrives within 20 ms. An arrival after 35 ms violates this assumption. Typical speed alone does not justify such a bound.
- **Asynchronous:** assume no delay bound for an Internet upload. A reply arriving after 2 s is permitted even if the client chose a 1 s timeout.
- **Partially synchronous:** congestion initially causes 2 s delays. After an unknown stabilization time, assume all new messages between healthy nodes arrive within 20 ms and this bound continues to hold. A brief period of fast responses is insufficient to establish that assumption.

These are hypothetical model choices, not measurements or claims about the production portal. A message-delay bound is only part of a response-time bound: processing and the return path also matter.

A deadline is a policy chosen by an application. It can limit how long a user waits even when it cannot prove that a remote node crashed. If the model supplies dependable bounds, stronger conclusions may be possible. Without those bounds, “usually fast” is not the same as “guaranteed to answer by this time.”

Why does “usually” not establish a guarantee? It allows exceptions, while a model guarantee must hold in every execution permitted by that model. Suppose replies usually arrive within 100 ms, but one takes 500 ms. Declaring the server crashed after 100 ms could mistake a slow server for a failed one. The values are illustrative. A correctness argument that depends on a 100 ms upper bound does not cover this slower execution; observations

of typical speed do not establish that bound. A model assumption is a basis for reasoning, not evidence that the real system always satisfies it.

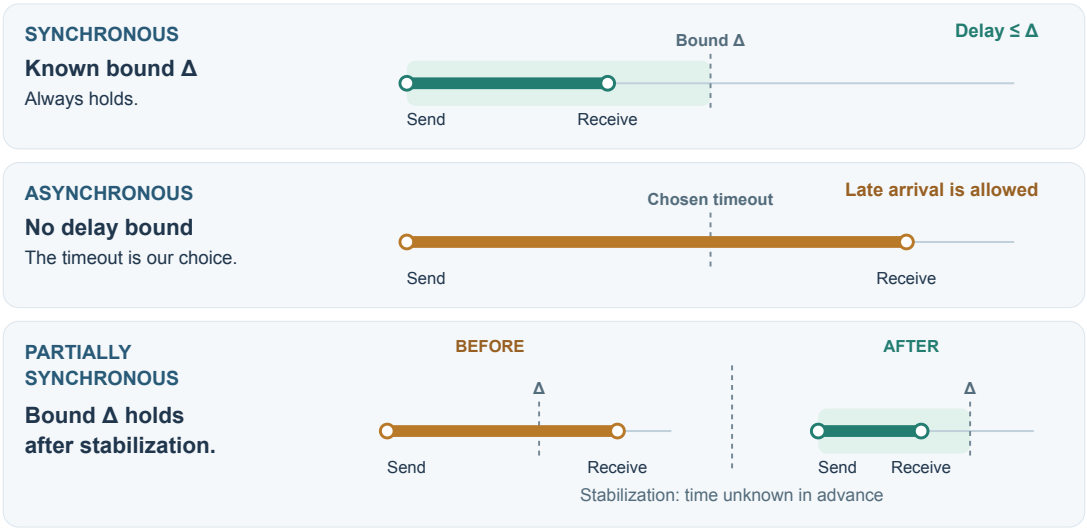


Figure 19: Send and receive times compared with a delay bound Δ. A chosen timeout is not a guarantee. In eventual synchrony, messages sent after an unknown stabilization time obey the bound.

11.6 Applying the vocabulary to real systems

These examples compare specific operations and configurations, rather than assign one universal guarantee to an entire product. A replica is another process maintaining a copy of data. A quorum here means a majority of the relevant members or replicas.

11.6.1 etcd: creating a key atomically

etcd is a replicated key-value store where distributed systems (e.g. Kubernetes) keep configuration and coordination data. Consider three members that follow the protocol (they do not lie) and can crash and recover with their durable protocol state intact. Communication may be delayed or partitioned. Raft preserves agreement despite these timing problems; progress requires a reachable majority and sufficiently stable communication and processing. In a $2 + 1$ partition, the two-member side can elect a leader and commit operations, while the isolated member cannot commit a competing history. Losing the majority stops writes rather than authorizing conflicting commits. These conditions follow the Raft model and etcd failure documentation.

Application invariant: at most one client successfully creates the key before that key is deleted. Implement the creation as one atomic transaction: compare that the key is absent, then create it. Assume no deletion, expiry or other code bypasses this rule. The invariant follows from the transaction's atomic ordering; separate read and write calls do not establish

it. This applies the etcd KV API guarantees. It does not grant exclusive control over an external machine or resource. As before, a timed-out client may not know whether its write was confirmed.

11.6.2 Cassandra: configuration changes the promise

Apache Cassandra is a distributed NoSQL database that keeps the same data on several servers. Take three replicas of a single cell and ordinary reads and writes at consistency level **ONE**. The coordinator needs one relevant replica to acknowledge an operation. With adequate resources, a client can therefore complete an operation without contacting a majority, but a later read from another replica may return old data. Replicas follow the protocol (they do not lie) and can crash and recover; delayed messages and partitions are expected. Timestamp-based reconciliation selects among conflicting values, so the selected value is not necessarily the last write in real-world time. See Cassandra's Dynamo architecture. The difference from etcd lies in when a write is confirmed: etcd confirms it only after a majority accepts it and assigns it a place in one order, so the minority side of a partition cannot write. Cassandra at ONE confirms after one replica and reconciles replicas later, so both sides of a partition can write, and timestamps resolve conflicts.

Application invariant not supplied: "sell a seat at most once" does not follow from ordinary read-then-write operations. Two clients can read "free" and both report a successful sale. Reconciling to one final stored value does not undo those two promises. Cassandra provides lightweight transactions for conditional operations with stronger consistency. Choosing QUORUM alone does not make separate reads and writes atomic.

Progress: an operation at ONE needs a responsive relevant replica; at QUORUM it needs two of the three. Neither works when no required replica can be reached. Eventual convergence needs recovered communication, retained updates and effective repair; with writes quiescent, replicas can converge after learning the same updates. Hints and repair explain why recovery is work the system and operators must perform, rather than something implied by the word "eventual". Permanent data loss or a permanent partition invalidates this convergence argument.

11.6.3 Bitcoin: validity and probabilistic settlement

Bitcoin coordinates a payment history among participants who may be malicious. Unlike the trusted-member crash model above, its security reasoning includes adversarial mining. It assumes secure cryptography, an honest majority of hash rate (honest miners hold most of the mining compute) and adequate block propagation relative to mining. Counting honest computers is not enough. Nodes validate transactions and select the valid chain with most accumulated proof of work. See the Bitcoin whitepaper, especially sections 5 and 11, and the developer guide.

Invariant: a transaction output cannot be spent twice within one valid chain. This does not imply that every node immediately agrees on one history: nodes can temporarily follow competing branches containing conflicting spends. **Settlement safety is probabilistic:**

additional confirmations reduce reversal risk under the security assumptions, rather than establish absolute finality.

Progress: mining and communication allow the chain to grow. Inclusion of a particular transaction also requires that it remain valid, reach miners and be selected; capacity, fees and miner policy matter. There is no fixed inclusion deadline. During a partition, separate branches can grow; after reconnection, switching to the selected chain can reverse a previously observed confirmation. A majority attacker can undermine settlement or censor transactions without being able to make an honest validating node accept an invalid signature.

These examples show three distinct promises: atomic key creation that stops without a majority; configurable replica acknowledgement and eventual reconciliation; and adversarial agreement with probabilistic settlement. Each still requires the application to state its own invariant and progress conditions.

11.7 Combining the assumptions for our portal

The model consists of reasonable assumptions that we, the portal's architects and developers, can anticipate: they describe the faults the design must handle, not facts about the system, and the design holds only while they hold. The dimensions form one model together. They are not successive levels a system must pass through.

- **Network:** requests and replies may be delayed, lost, repeated, or reordered; no message tampering is included.
- **Nodes:** services can crash and restart. Accepted records in durable storage (disks in RAID) survive the modelled process crashes; state in main memory (RAM) is lost. This is an assumption about storage, not a promise that disks cannot fail in reality.
- **Timing:** there is no guaranteed maximum response time.

The desired safety claim is that retries never create two accepted entries for the same intended submission, including across restarts. The implementation must justify that claim: durable records alone do not coordinate duplicate detection with applying an effect.

For eventual acceptance, additionally assume that the required services eventually remain available, communication eventually succeeds, pending work is retained or retried, and processing eventually runs. A permanent partition violates these progress conditions but must not permit duplicate acceptance. Eventual synchrony may be needed by a particular algorithm's progress argument; it is not required for every possible form of eventual progress.

11.8 Optional reading: system models in practice

Deployment location does not uniquely determine a system model. For a **WWW application**, a useful starting point is asynchronous delays, disconnections, and crash-recovery for trusted servers; clients may be untrusted and behave arbitrarily. A backend correctness

claim must specify which participants it covers. Untrusted clients alone do not imply that the backend needs Byzantine consensus.

For an API and worker deployed on **one computer**, model separate processes using IPC or loopback, possible process restarts, and correlated failure when the host fails. An ordinary operating system does not provide a hard response-time bound merely because communication is local. This deployment does not provide tolerance of physical host failure.

For services across a **LAN**, low typical latency does not eliminate scheduling pauses, congestion, or network partitions. Asynchronous timing and crash-recovery for trusted servers are a reasonable starting point. Where an algorithm requires stable timing to make progress, state eventual synchrony and recovery as additional assumptions. This also applies to Web services. Guaranteed synchrony requires justified network and processing bounds, rather than simply a nearby server. These are example modelling choices using the distinctions in [3, sections 1.1 and 2.3].

12 Exercises

1. **Identify the boundary.** In the submission diagram, list each participant and one item of state it owns. Which machine failures could affect more than one participant if two services share a host?
2. **Classify an execution.** Worker W executes A1, B1, A2, B2. Are A and B concurrent? Does this trace demonstrate parallelism? Explain the distinction.
3. **Interpret evidence.** A submission request times out after two seconds. Give two different server-side histories consistent with that observation. Does a longer timeout remove the ambiguity in a model without a known upper delay bound?
4. **Audit an assumption.** A design says, "The assignment service usually responds within 100 ms, so after 101 ms it must have crashed." Identify the missing guarantee and propose a more accurate statement.

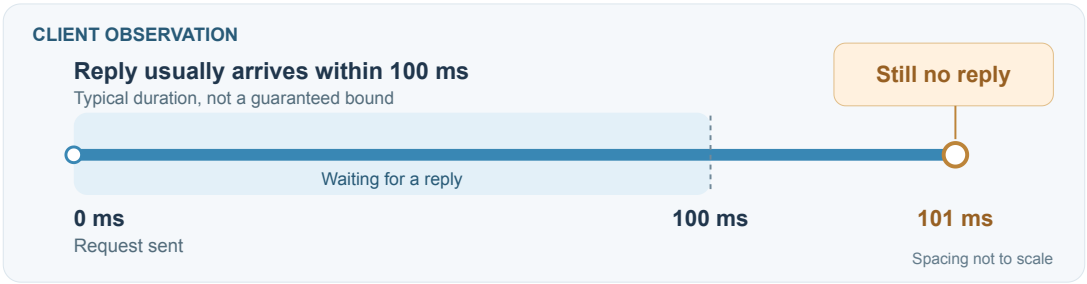


Figure 20: The client's observation at 101 ms. The shaded region marks usual response time, not a guaranteed bound; the service's actual state is not shown. Time moves right and spacing is not to scale.

5. **Protect a rule.** Two attempts of the same intended submission both check for an existing accepted entry before either creates one. Write an interleaving that produces two entries, state the violated invariant, and explain why adding a second server does not solve the issue by itself.
6. **Choose a tradeoff.** The assignment service is unreachable but cached assignment instructions exist. Discuss when showing those instructions is acceptable and why accepting a new submission may need a stronger guarantee.
7. **State a model.** For the portal, state one network, node, and timing assumption. Name a rule to preserve and an additional condition needed for eventual acceptance. What changes during a permanent partition?
8. **Homework: sequential invoice numbers.** A company issues invoices from several servers, and invoice numbers must be sequential: 1, 2, 3, … with no repeats and no gaps. State the invariant. Two servers issue an invoice at the same moment: who assigns the number? A server takes number 42 and crashes before storing the invoice: what happens to 42? A till times out and retries: how do you avoid issuing two invoices? Finally, is there a design that gives up nothing—availability, speed, or strict global sequence? If not, decide what you would give up and justify the choice.

12.1 Suggested reasoning

1. The client owns its current view and pending request; submission owns processing state; the assignment service owns deadlines and file rules; submission storage owns complete project files and accepted-submission records. A shared host creates a shared failure domain. Exact ownership can differ in a real design and should be stated.
2. The tasks overlap in progress, so the execution is concurrent. The single-worker trace does not show simultaneous execution and therefore does not demonstrate parallelism.
3. The request could be lost, or it could be applied with its reply lost. A larger finite deadline changes the waiting policy but does not supply a bound the model does not guarantee.
4. An observed typical delay is not a guaranteed maximum including network and processing time. “No response by the deadline; suspect unavailability and begin the defined recovery policy” is a defensible observation.
5. A finds no entry, B finds no entry, A creates an accepted entry, then B creates another. The one-entry-per-intended-submission invariant fails. Correctness needs a mechanism that coordinates the check and effect; merely duplicating an uncoordinated implementation leaves the problem.
6. Stale assignment instructions may be acceptable if labelled appropriately. Accepting a submission can depend on the current deadline and permitted file types and sizes. The decision follows the operation’s requirements and any explicitly permitted staleness.

7. One possible model allows loss and reordering, crash-recovery with durable accepted records, and unbounded response delays. Preserve the no-duplicate rule. Eventual acceptance additionally needs recovery, eventual communication, retained or retried work, and processing. A permanent partition defeats the communication condition for progress; it does not relax safety. Other explicit, consistent choices are acceptable.

8. The invariant has two parts: no number is used twice (safety against duplicates) and no number is skipped (no gaps). A single counter service satisfies both only while it is reachable; it becomes a bottleneck and a single point of failure. A crash after assigning 42 but before storing the invoice creates a gap unless assignment and storage are made atomic, or 42 is recorded as void. A retry after a timeout creates a duplicate unless the request carries an identifier and the server returns the existing invoice for a repeated identifier (idempotency, as in Web Apps). Giving each server its own range keeps numbers unique and available but gives up the global sequence. Numbering separately per till or premises keeps a strict sequence within each, which is how many fiscal systems label invoices. No design gives up nothing: a gapless global sequence requires coordination, so during a partition or a coordinator crash the system must either wait, losing availability, or continue without coordination, losing the sequence. Replication (Lecture 06), consensus (Lecture 07) and transactions (Lecture 09) develop the coordinated options.

13 Recap

13.1 Hidden assumptions to challenge

The classic fallacies of distributed computing warn against silently assuming reliable delivery, zero latency, unlimited bandwidth, inherent security, unchanging topology, one administrative authority, free data transport and uniform components. The practical response is to state the actual conditions and decide how to handle their violation. For example, a lost reply challenges reliable delivery, while a slow reply challenges the idea of negligible latency. Multiple teams may impose different operational policies; compatible interfaces must be designed despite different versions and implementations. This does not forbid stronger abstractions: it requires explaining how they are provided and where their guarantees stop. See Oracle's discussion.



Figure 21: A reasoning checklist: identify participants and messages, weigh benefits and costs, consider failures, state assumptions and invariants, then specify conditions for progress.

Distributed components cooperate through communication while retaining independent execution and local state. Distribution can meet useful requirements, but it creates coordination and recovery work. Concurrency does not require parallel hardware. A partial failure can leave some processes alive while an operation becomes unavailable. Most importantly, a timeout does not by itself determine a remote operation's outcome.

For any proposed design, ask: **who participates, what messages cross boundaries, what must remain true, and which assumptions justify the proposed behaviour?**

13.2 Design the next step: did I submit on time?

The deadline is **midnight**. A student's upload finishes at **23:59**, but the checks finish at **00:19**. **How would you design the portal so the student knows whether they submitted on time—even after closing the browser?** Propose what the portal should save and show, a rule to preserve, and your network, restart, and timing assumptions.

First state the deadline policy: which event counts as submission? The browser reporting upload completion is not by itself evidence that the service has durably received the project. Distinguish receiving a project before the deadline from deciding whether it passes the checks.

Directions to explore: a receipt for a stored upload, a server-recorded receipt time, a "received, checks pending" status, and an identifier that lets the student return to the same submission later. Explain what each part of your proposal establishes. More than one design can be defensible; do not assume that a receipt proves the project passed validation.

Extend your design: what if the server crashes during the checks? Which information must survive, how does processing resume, and how can the student retry without creating

another submission? These questions lead into later lectures on service boundaries, durable state, and safe retries.

14 Sources and further reading

[1] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems, 4th edition*. 2023. Optional broad textbook reading. The authors offer a free personalized PDF through an email-and-CAPTCHA request form; the download link arrives by email. The directly accessible notes in [3] cover the required introductory topics.

[2] Jim Waldo, Geoff Wyant, Ann Wollrath, and Sam Kendall. *A Note on Distributed Computing*. Sun Microsystems Laboratories, 1994. Public author page with the paper attached. Read for the distinction between local and remote computation, including latency, concurrency, and partial failure.

[3] Martin Kleppmann. *Distributed Systems*. University of Cambridge, 2021/22. Public lecture notes, published under CC BY-SA. Read sections 1.1–1.3 for foundations and RPC, 2.1 for uncertainty, and 2.3 for node, network, and timing models. Section 5.1 provides optional follow-up on retries and duplicate effects.

[4] Jeffrey Dean and Luiz André Barroso. *The Tail at Scale*. Communications of the ACM 56(2), 2013. How rare slow replies dominate latency when a request waits for many services.